24K-0764
24K-0826
24K-0527

DAA Project

Milestone #1

Q1(d)

The running time of quicksort is dictated by the distribution of its partition sizes which in turn depends on choice of 'pivot'

WORST-CASE

Occurs when when partition step consistently yields most unbalanced split possible ; one subproblem of size 0 and other of size $n-1$. This happens when input array is already sorted.

$$C_{worst}(n) = (n+1) + (n) + \cdots + 3 = \Theta(n^2).$$

BEST-CASE

It occurs when partition step consistently splits the array into two virtually equal halves at every level of recursion. This happens when Pivot is the median of current sub array.

$$C_{best}(n) = 2C_{best}(n/2) + n$$

Solving with Master Theorem Yields:

$$C_{best}(n) \in \Theta(n \log_2 n)$$

24K-0764.
24K-0826
24K-0527

Q1(c) We mirror the structure of classical 2D (a) closest-pair algorithm.

1.) **Divide**: Sort the initial array once as a pre-processing step (or we assume) it as part of setup. Find middle element index $m = \lfloor \frac{n}{2} \rfloor$. Split the sorted array into two halves: a left subset S_L and right subset S_R

2.) **Conquer**: Recursively find the closest-pair distance δ_L in S_L and δ_R in S_R

3.) **Combine**: The true min distance δ is either δ_L, δ_R, or a cross-boundary distance between an element in S_L and an element in S_R. Since array is sorted, the only candidate pair crossing boundary (dividing Line) consists of the rightmost element of S_L and left most element of S_R. We check this single cross boundary distance, update δ if it is smaller, and return it.

COMPLEXITY:

excluding initial sorting:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$$

$$T(n) \in \Theta(n)$$

Total Algorithmic complexity:

$$T_{total}(n) = T_{sort}(n) + T_{recursive}(n)$$
$$= \Theta(n \log n) + \Theta(n)$$
$$T_{total}(n) \in \Theta(n \log n) \text{ efficiency class!}$$

Q1(e)

(b) While simple linear scan over a sorted array achieves the same time complexity $O(n)$ with better space complexity $O(1)$ v/s $O \log(n)$ stack space, the recursive 1D_Closest-PAIR algorithm is valuable from architectural standpoint.

∝ ——— ∝

Q1(f) Let $A[1...n]$ be an array of distinct real Numbers. The sequence is unimodal.

$$A[1] < A[2] < \ldots < A[p-1] < A[p] > A[p+1] \ldots > A[n]$$

Constraint: $O(\log(n))$ array entry reads.

Algorithm:-

Because the array cannot be randomized & maintains structural monotonicity on both sides of P. So $O(n)$ is sub-optimal.

Let the search space be bounded by [low, high]

So, $mid = \left\lfloor \frac{low + high}{2} \right\rfloor$

we evaluate the local neighbours (immediate) i.e A[mid-1] & A[mid+1]. The relative order yields 03 Mutually exclusive cases:

CASE 1:

$A[mid-1] < A[mid] < A[mid+1]$

then 'mid' entry must come strictly before 'p' and so we can continue recursively on entries $n/2 + 1$ through n

CASE 2:

$A[mid-1] > A[mid] > A[mid+1]$

then mid entry 'mid' must come strictly after 'p' & so we can continue recursively on entries 1 through $n/2 - 1$.

CASE 3:

$A[mid-1] < A[mid] > A[mid+1]$.

we're done; peak entry is equal to 'mid' entry in this case.

* Each recursive call performs constant number of operations.

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

Master Theorem $\Rightarrow T(n) \in \Theta(\log n)$

DAA project

Date:

24k-0527
24k-0826
24k-0764

Q No. 1

a)

1)

```
minmax (arr[l...r])
    if (l==r)
        return (arr[l], arr[l])
    if (r==l+1)
        if (arr[l] < arr[r])
            return (arr[l], arr[r])
        else
            return (arr[r], arr[l])
    mid = ⌊(l+r)/2⌋

(min1, max1) = minmax(arr[l...mid])
(min2, max2) = minmax(arr[mid...r])

return (min(min1, min2), max(max1, max2))
```

2)

$C(n) = 2C\left(\frac{n}{2}\right) + 2$ where $C(2) = 1$

By iterative method

$C(n) = 2C\left(\frac{n}{2}\right) + 2$ ---(i)

$C\left(\frac{n}{2}\right) = 2C\left(\frac{n}{4}\right) + 4$

CamScanner

Date:

```
else
    M.add(R[j]); j++
count += count + (L.size - i)

for (i ≤ L.size)
    M.add(L[i])
for (j ≤ R.size)
    M.add(R[j])

return (M, count)
```

Recurrance Relation

$$T(n) = 2T\left(\frac{n}{2}\right) + n \qquad T(1) = 0$$

$$T(n) = \theta(n \log n)$$

CamScanner

Date:

$$C(n) = 2^2 C\left(\frac{n}{4}\right) + 4 + 2$$

$$C(n) = 2^3 C\left(\frac{n}{8}\right) + 8 + 4 + 2$$

$$C(n) = 2^k C\left(\frac{n}{2^k}\right) + 2(2^k - 1)$$

As $C(2) = 1$

$$2 = \frac{n}{2^k}$$

$$n = 2^{k+1}$$

$$\log_2 n = k + 1$$

$$k = \log_2 n - 1$$

$$C(n) = 2^{\log_2 n - 1} C(2) + 2(2^{\log_2 n - 1} - 1)$$

$$= \frac{n}{2}(1) + 2\left(\frac{n}{2} - 1\right)$$

$$= \frac{n}{2} + \frac{2n}{2} - 2$$

$$= \frac{n + n - 2}{2}$$

$$C(n) = \frac{n + 2n - 4}{2} = \frac{3n}{2} - 2$$

$$\boxed{C(n) = \frac{3n - 2}{2}}$$

CamScanner

Date:

3.

Brute force scans current element with current min and max once each hence $2n-2$ comparisions.

Though both are $\theta(n)$. This divide-and-conquer approach decreases 25% comparisions.

b)

1)

```
calc_power (a , n)
    if (n == 1)
        return a
    h = calc_power (a, ⌊n/2⌋)
    if (n % 2 == 0)
        return (h * h)
    else
        return (h * h * a)
```

2) for exact number of multiplications

$$M(n) = M\left(\frac{n}{2}\right) + 1$$

By Master's theorem.

$a = 1 \quad b = 2 \quad f(n) = \theta(1) \quad d = 0$

$a = b^d$

$1^0 = 2^0$

$1 = 1$

$$M(n) = \theta(n^d \log_b n) = \theta(\log_2 n)$$

ijazi

CamScanner

Date:

3)

Brute force uses n-1 multiplications.

$\Theta(n)$

Divide and Conquer uses $\log_2 n$ multiplications

$\Theta(\log_2 n)$

for n = 1024,

Brute force does 1023 multiplications

DaC does 10 multiplications.

c)

Pseudo code:

```
count_inversions(& arr[l...r])
    if (l==r)
        return 0(arr, 0)
(L, x) = count_inversions(arr[l... mid])
(R, xy) = count_inversions(arr[mid.. r])
(M, z) = merge count (L, R)
return (M, x+y+z)

merge count (L, R)
  i=0, j=0, count=0
while (i < L.size && j < R.size)
    if (L[i] ≤ R[j])
        M.add(L[i]); i++
```

ijazi

Page No.